

Einsum Trees

An Abstraction for Optimizing the Execution of Tensor Expressions

Breuer · Blacher · Engel · Giesen · Heinecke · Klaus · Remke

CS259::Paul Talma

April 21, 2026

Overview

Einsum Notation

Tensor Processing Primitives

Einsum Trees

Results

Limitations and Questions

Einsum Notation

Einsum notation is a compact notation for **pure tensor operations**.

Canonical form:

$$\text{einsum}(l_1, \dots, l_n \rightarrow O; T_1, \dots, T_n)$$

Einsum expressions are composed of **input tensors** $T_1, \dots, T_n$, **input indices**¹ $l_1, \dots, l_n$, and **output indices** O .

¹Technically, *multi-indices*.

Einsum Semantics

Some examples:

Expression	Operation	Name
<code>einsum</code> ($pq, qr \rightarrow pr; A, B$)	$O_{pr} = \sum_q A_{pq} B_{qr}$	Matrix multiply AB
<code>einsum</code> ($pq, pq \rightarrow pq; A, B$)	$O_{pq} = A_{pq} B_{pq}$	Hadamard product $A \odot B$
<code>einsum</code> ($pq, qr \rightarrow r; A, B$)	$O_r = \sum_{pq} A_{pq} B_{qr}$	$= \sum_q B_{qr} \sum_p A_{pq}$

Interpretation:

- The output indices define the output dimensions.
- The input indices that do not appear in the output are **contracted**.
 - Contraction: multiply-then-sum.

Advantages of Einsum

Concise
Literate
Efficient
Easy to use

```
class ScaledDotProductAttention(nn.Module):
    """ Scaled Dot-Product Attention """
    def __init__(self, temperature, attn_dropout=0.1):
        super().__init__()
        self.temperature = temperature
        self.dropout = nn.Dropout(attn_dropout)
        self.softmax = nn.Softmax(dim=-2)

    def forward(self, q, k, v, mask=None):
        attn = torch.bmm(q, k.transpose(2, 1))
        attn = attn / self.temperature

        if mask is not None:
            attn = attn.masked_fill(mask, -np.inf)

        attn = self.softmax(attn)
        attn = self.dropout(attn)
        output = torch.bmm(attn, v)

        return output, attn


class MultiHeadAttentionOld(nn.Module):
    """ Multi-Head Attention module """
    def __init__(self, n_head, d_model, d_k, d_v, dropout=0.1):
        super().__init__()

        self.n_head = n_head
        self.d_k = d_k
        self.d_v = d_v

        self.w_qs = nn.Linear(d_model, n_head * d_k)
        self.w_ks = nn.Linear(d_model, n_head * d_k)
        self.w_vs = nn.Linear(d_model, n_head * d_v)
        nn.init.normal_(self.w_qs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_ks.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_vs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_v)))

        self.attention = ScaledDotProductAttention(temperature=np.power(d_k, 0.5))
        self.layer_norm = nn.LayerNorm(d_model)

        self.fc = nn.Linear(n_head * d_v, d_model)
        nn.init.xavier_normal_(self.fc.weight)

        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k, d_v, n_head = self.d_k, self.d_v, self.n_head

        sz_b, len_q, _ = q.size()
        sz_b, len_k, _ = k.size()
        sz_b, len_v, _ = v.size()

        residual = q

        q = self.w_qs(q).view(sz_b, len_q, n_head, d_k)
        k = self.w_ks(k).view(sz_b, len_k, n_head, d_k)
        v = self.w_vs(v).view(sz_b, len_v, n_head, d_v)
        q = q.permute(2, 0, 1, 3).contiguous().view(-1, len_q, d_k) # (n*nb) x lq x dk
```

```
class MultiHeadAttentionNew(nn.Module):
    def __init__(self, n_head, d_model, d_k, d_v, dropout=0.1):
        super().__init__()
        self.n_head = n_head

        self.w_qs = nn.Linear(d_model, n_head * d_k)
        self.w_ks = nn.Linear(d_model, n_head * d_k)
        self.w_vs = nn.Linear(d_model, n_head * d_v)

        nn.init.normal_(self.w_qs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_ks.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_vs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_v)))

        self.fc = nn.Linear(n_head * d_v, d_model)
        nn.init.xavier_normal_(self.fc.weight)
        self.dropout = nn.Dropout(dropout)
        self.layer_norm = nn.LayerNorm(d_model)

    def forward(self, q, k, v, mask=None):
        residual = q
        q = rearrange(self.w_qs(q), 'b l (head k) -> head b l k', head=self.n_head)
        k = rearrange(self.w_ks(k), 'b t (head k) -> head b t k', head=self.n_head)
        v = rearrange(self.w_vs(v), 'b t (head v) -> head b t v', head=self.n_head)
        attn = torch.einsum('hbik,hbtk->hbtl', [q, k]) / np.sqrt(q.shape[-1])
        if mask is not None:
            attn = attn.masked_fill(mask[None], -np.inf)
        attn = torch.softmax(attn, dim=-1)
        output = torch.einsum('hbtl,hbtlv->hbtlv', [attn, v])
        output = rearrange(output, 'head b l v -> b l (head v)')
        output = self.dropout(self.fc(output))
        output = self.layer_norm(output + residual)
        return output, attn
```

Einsum Implementation

Matrix multiply:

$$C = \text{einsum}(pq, qr \rightarrow pr; A, B)$$

- C has dimensions $|p| \times |r|$
- $C_{pr} = \sum_q A_{pq} B_{qr}$

```
1 for p in range(P):  
2     for r in range(R):  
3         for q in range(Q):  
4             C[p,r] += A[p,q]*B[q,r]
```

Einsum Implementation

Matrix multiply:

$C = \text{einsum}(pq, qr \rightarrow pr; A, B)$

- C has dimensions $|p| \times |r|$
- $C_{pr} = \sum_q A_{pq} B_{qr}$

```
1 for p in range(P):
2     for r in range(R):
3         for q in range(Q):
4             C[p,r] += A[p,q]*B[q,r]
```

Batched matrix multiply + reduce:

$C = \text{einsum}(bpq, bqr \rightarrow br; A, B)$

- C has dimensions $|b| \times |r|$
- $C_{br} = \sum_{pq} A_{bpq} B_{bqr} = \sum_q B_{bqr} \sum_p A_{bpq}$

```
1 for b in range(B):
2     for r in range(R):
3         for p in range(P):
4             for q in range(Q):
5                 C[b,r] += A[b,p,q]*B[b,q,r]
```


Multi-Tensor Einsum Implementation

$D = \text{einsum}(pq, qr, rs \rightarrow ps; A, B, C)$

- $D_{ps} = \sum_{qr} A_{pq} B_{qr} C_{rs}$

```
1 for p in range(P):  
2     for s in range(S):  
3         for q in range(Q):  
4             for r in range(R):  
5                 D[p,s] += A[p,q]*B[q,r]*C[r,s]
```

Multi-Tensor Einsum Implementation

$D = \text{einsum}(pq, qr, rs \rightarrow ps; A, B, C)$

- $D_{ps} = \sum_{qr} A_{pq} B_{qr} C_{rs}$

```
1 for p in range(P):  
2     for s in range(S):  
3         for q in range(Q):  
4             for r in range(R):  
5                 D[p,s] += A[p,q]*B[q,r]*C[r,s]
```

The naive approach is inefficient! This is $O(|p| \cdot |q| \cdot |r| \cdot |s|) \approx O(n^4)$.

Multi-Tensor Einsum Implementation

$\text{einsum}(pq, qr, rs \rightarrow ps; A, B, C) = \text{einsum}(pr, rs \rightarrow ps; AB, C)$,
where $AB = \text{einsum}(pq, qr \rightarrow pr; A, B)$.

- $D_{ps} = \sum_r (AB)_{pr} C_{rs}$

```
1 for p in range(P):
2     for r in range(R):
3         for q in range(Q):
4             AB[p,r] += A[p,q]*B[q,r]
5
6 for p in range(P):
7     for s in range(S):
8         for r in range(R):
9             D[p,s] += AB[p,r]*C[r,s]
```

Multi-Tensor Einsum Implementation

$\text{einsum}(pq, qr, rs \rightarrow ps; A, B, C) = \text{einsum}(pr, rs \rightarrow ps; AB, C)$,
where $AB = \text{einsum}(pq, qr \rightarrow pr; A, B)$.

- $D_{ps} = \sum_r (AB)_{pr} C_{rs}$

```
1 for p in range(P):
2     for r in range(R):
3         for q in range(Q):
4             AB[p,r] += A[p,q]*B[q,r]
5
6 for p in range(P):
7     for s in range(S):
8         for r in range(R):
9             D[p,s] += AB[p,r]*C[r,s]
```

This is $O(|p| \cdot |r| \cdot |q| + |p| \cdot |s| \cdot |r|) \approx O(n^3)$.

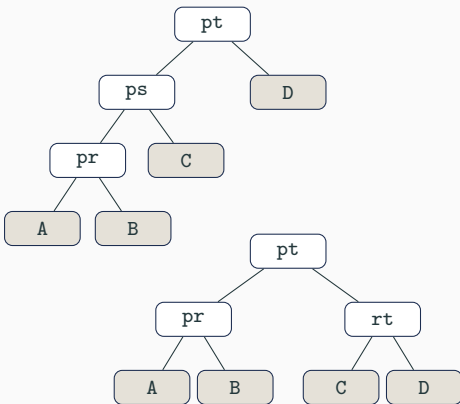
Contraction Paths

Key idea: implement arbitrary einsum expressions by reducing them to binary einsums.

New problem: there are many possible reductions; some are more efficient; finding optimal reduction is NP-hard.

- Cost models: max size of intermediate tensor; number of FLOPs.
- Despite NP-hardness, good heuristics exist.

`einsum(pq, qr, rs, st → pt; A, B, C, D)`



Contraction trees

Tensor Processing Primitives

Implementing Binary Einops

Once we've decided on a contraction path, we must implement the binary einops.

Typically, this means executing matrix multiplies.

And typically, *that* means calling an optimized GEMM (GEneral Matrix Multiply) kernel from BLAS.²

- The paper also uses packed GEMM (batched MM) and transpose.
- GEMM, packed GEMM, and transpose form the paper's **Tensor Processing Primitives**, into which einsum computations decompose.

²Basic Linear Algebra Subprograms, a library of optimized LA primitives written in Fortran.

GEMM and Memory Layout

GEMM assumes a specific data layout.

Each problem dimension is classified into one of the basic GEMM types: M, N, K, C, R.

- M/N: free dimensions preserved in output (present in single input and in output)
- C: batch dimensions (in several inputs and in output)
- K: reduced dimensions (in several inputs but not output; requires multiplication)
- R: reduced dimensions (in single input and not output; only requires sum)

GEMM and Memory Layout

A GEMM call has the form $C[M][N] += A[M][K] * B[K][N]$.

A packed GEMM call has the form $C[C][M][N] += A[C][M][K] * B[C][K][N]$.

BLAS³ requires the N and K dimensions to be contiguous in memory (i.e. to have stride 1).

- This enables SIMD vectorization and plays nicely with cache behavior and prefetching.

Thus, BLAS constrains the einsum dimensions to the format

$$(C)MK, (C)KN \rightarrow (C)MN.^4$$

³Basic Linear Algebra Subprograms, which implements GEMM.

⁴Note: The paper uses $KMC, NKC \rightarrow NMC$ because BLAS is written in Fortran, which follows a column-major data layout. I've translated the requirement to a more C-friendly layout.

GEMM and Memory Layout

BLAS requires $\text{CMK}, \text{CKN} \rightarrow \text{CMN}$.

$\text{einsum}(b_{ij}, b_{jk} \rightarrow b_{ik}; A, B)$

Dimension	GEMM Type
b	C
i	M
j	K
k	N

Order: $\text{CMK}, \text{CKN} \rightarrow \text{CMN}$.

$\text{einsum}(a_b, a_c \rightarrow c_b; A, B)$

Dimension	GEMM Type
a	K
b	M
c	N

Order: $\text{KM}, \text{KN} \rightarrow \text{NM}$.

GEMM and Memory Layout

BLAS requires $\text{CMK}, \text{CKN} \rightarrow \text{CMN}$.

$\text{einsum}(b_{ij}, b_{jk} \rightarrow b_{ik}; A, B)$

Dimension	GEMM Type
b	C
i	M
j	K
k	N

Order: $\text{CMK}, \text{CKN} \rightarrow \text{CMN}$.

$\text{einsum}(ab, ac \rightarrow cb; A, B)$

Dimension	GEMM Type
a	K
b	M
c	N

Order: $\text{KM}, \text{KN} \rightarrow \text{NM}$.

The second example can be brought into GEMM-friendly layout by transposing A and the output: $\text{einsum}(ba, ac \rightarrow bc; A^\top, B)$. But this involves a potentially costly transpose on A .

Einsum Trees

To solve the problem, the paper

- i) enriches the representation of the contraction path into an *einsum tree*, and
- ii) systematically transforms the tree to make each binary einop GEMM-friendly.

The enriched representation is the **einsum tree**.

- Functions as an Intermediate Representation (IR), facilitating the analysis and optimization of the original einsum expression.
- Build upon *contraction trees*, which simply record data dependencies among binary einsums.
- Einsum trees augment contraction trees with information about child node ordering and about dimension order within a node.

Three transformations:

1. Swap children of a node
2. Reorder indices of a node
3. Insert a permutation node

Any einsum tree can be transformed into a GEMM-compatible one by applying these transformations.

Step 1: assign a GEMM type to each problem dimension

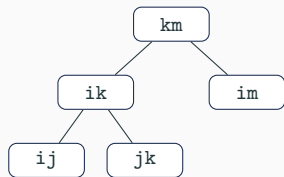
Step 2: transform einsum tree to make it GEMM-friendly:

- If I_1 has rightmost N dimension and I_2 has rightmost M dimension, swap them.
- Reorder indices within nodes to satisfy GEMM requirements (**CMK** for left children, **CKN** for right children).
- Insert permutation nodes below reordered nodes.

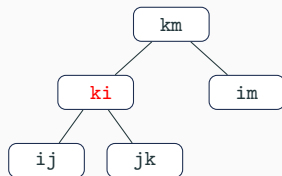
Example

$\text{einsum}(ij, jk, im \rightarrow km)$

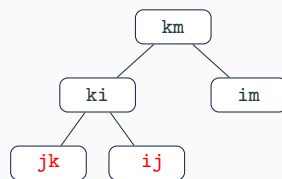
Contraction tree:



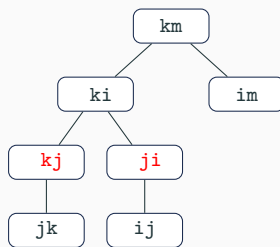
Reorder indices:



Swap children:



Insert permutation:



Results

Evaluations

Comparison to: TVM-Ansor, ATen (pytorch), OE-Torch on TCCG benchmark.

ID	Contraction	Dimension Sizes	Ari. Int.
1	abdfc,cf→abcde	48,36,24,36,48,36	7
2	acdfe,bf→abcde	48,24,36,36,48,36	7
3	abed,ce→abcd	96,84,24,96,96	10
4	abcfe,df→abcde	48,36,36,24,48,48	8
5	aced,be→abcd	96,24,84,96,84	9
6	jki,efghjk→efghi	6,64,6,64,24,6,64	11
7	abed,ce→abcd	96,84,24,84,96	10
8	bcbf,adeg→abcdef	24,20,20,24,20,20,24	12
9	bdgf,aceg→abcdef	24,20,20,24,20,20,24	12

Binary Contractions

ID	Einsum String / Dim. Sizes / Contraction Path
SYN	iaje,bf,dcba,cigj,dh→hgfei
	24,48,12,56,32,64,8,84,8,72 (1,2),(2,3),(0,1),(0,1)
TT	af,fbg,gch,hdi,ie→abcde
	100,72,128,128,3,71,305,32,3 (1,2),(0,3),(0,1),(0,1)
FCTN	aefg,behi,cfhj,dgij→abcd
	60,60,20,20,8,8,8,8,8 (2,3),(0,2),(0,1)

Contraction Trees

FLOPs

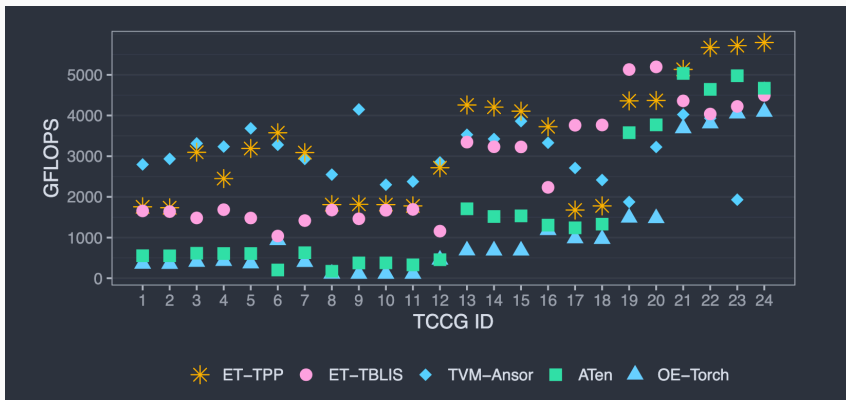


Table 7. Compilation time for ET-TPP, ET-TBLIS, TVM-Ansor, and OE-Torch. All measurements were taken on the Grace system.

ID	ET-TPP	ET-TBLIS	TVM-Ansor	OE-Torch
SYN	0.78 ms	0.54 ms	37.83 m	0.17 ms
TT	2.16 ms	0.55 ms	66.92 m	0.17 ms
FCTN	0.64 ms	0.41 ms	25.52 m	0.15 ms
TW	0.85 ms	0.47 ms	42.17 m	0.18 ms
GETD	0.85 ms	0.39 ms	39.68 m	0.17 ms
TRN	0.78 ms	0.50 ms	N/A	0.26 ms
MERA	3.84 ms	1.26 ms	N/A	0.64 ms
TNLM	8.79 ms	1.26 ms	N/A	1.37 ms

Summary of Results

- Comparable or superior performance on most benchmark problems.
- Vastly better compilation time than TVM-Ansor.

Limitations and Questions

Limitations and Questions

1. The optimization algorithm is presented without any analysis. Is it optimal? Does it fail on certain cases?
2. Contraction paths are given. But perhaps contraction path and layout optimization need to be jointly optimized?
3. Some liberties taken with the benchmark.
4. CPU-only.